

OSGi Working Group
OSGi Specification repo
for [org.osgi.service.net-](https://org.osgi.service.net-adapter/)
org.osgi.service.net-adapter

September 2026



LICENSE

Eclipse Foundation Specification License – v1.0

By using and/or copying this document, or the Eclipse Foundation document from which this statement is linked, you (the licensee) agree that you have read, understood, and will comply with the following terms and conditions:

Permission to copy, and distribute the contents of this document, or the Eclipse Foundation document from which this statement is linked, in any medium for any purpose and without fee or royalty is hereby granted, provided that you include the following on ALL copies of the document, or portions thereof, that you use:

- link or URL to the original Eclipse Foundation document.
- All existing copyright notices, or if one does not exist, a notice (hypertext is preferred, but a textual representation is permitted) of the form: "Copyright © [date-of-document] Eclipse Foundation, Inc. <<url to this license>>"

Inclusion of the full text of this NOTICE must be provided. We request that authorship attribution be provided in any software, documents, or other items or products that you create pursuant to the implementation of the contents of this document, or any portion thereof.

No right to create modifications or derivatives of Eclipse Foundation documents is granted pursuant to this license, except anyone may prepare and distribute derivative works and portions of this document in software that implements the specification, in supporting materials accompanying such software, and in documentation of such software, PROVIDED that all such works include the notice below. HOWEVER, the publication of derivative works of this document for use as a technical specification is expressly prohibited.

The notice is:

"Copyright © [date-of-document] Eclipse Foundation. This software or document includes material copied from or derived from [title and URI of the Eclipse Foundation specification document]."

Disclaimers

THIS DOCUMENT IS PROVIDED "AS IS," AND THE COPYRIGHT HOLDERS AND THE ECLIPSE FOUNDATION MAKE NO REPRESENTATIONS OR WARRANTIES, EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, NON-INFRINGEMENT, OR TITLE; THAT THE CONTENTS OF THE DOCUMENT ARE SUITABLE FOR ANY PURPOSE; NOR THAT THE IMPLEMENTATION OF SUCH CONTENTS WILL NOT INFRINGE ANY THIRD PARTY PATENTS, COPYRIGHTS, TRADEMARKS OR OTHER RIGHTS.

THE COPYRIGHT HOLDERS AND THE ECLIPSE FOUNDATION WILL NOT BE LIABLE FOR ANY DIRECT, INDIRECT, SPECIAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF ANY USE OF THE DOCUMENT OR THE PERFORMANCE OR IMPLEMENTATION OF THE CONTENTS THEREOF.

The name and trademarks of the copyright holders or the Eclipse Foundation may NOT be used in advertising or publicity pertaining to this document or its contents without specific, written prior permission. Title to copyright in this document will at all times remain with copyright holders.

Preface

Implementation Requirements

An implementation of a Specification: (i) must fully implement the Specification including all its required interfaces and functionality; (ii) must not modify, subset, superset or otherwise extend the OSGi Name Space, or include any public or protected packages, classes, Java interfaces, fields or methods within the OSGi Name Space other than those required and authorized by the Specification. An implementation that does not satisfy limitations (i) (ii) is not considered an implementation of the Specification and must not be described as an implementation of the Specification. "OSGi Name Space" shall mean the public class or interface declarations whose names begin with "org.osgi" or any recognized successors or replacements thereof. An implementation of a Specification must not claim to be a compatible implementation of the Specification unless it passes the Technology Compatibility Kit ("TCK") for the Specification.

Feedback

This specification can be downloaded from the OSGi Documentation web site:

<https://docs.osgi.org/specification/>

Comments about this specification can be raised at:

<https://github.com/osgi/osgi/issues>

Table of Contents

1	Introduction	5
1.1	Reader Level	5
1.2	Version Information	5
1.3	References	5
143	Network Interface Information Service Specification	7
143.1	Introduction	7
143.2	NetworkAdapter Service	8
143.3	NetworkAddress Service	10
143.4	A Controller Example	11
143.5	Security	12
143.6	org.osgi.service.networkadapter	12
143.7	References	18

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT

1 Introduction

This document contains a specification developed by the OSGi working group.

1.1 Reader Level

This specification is written for the following audiences:

- Application developers
- Framework and system service developers (system developers)
- Architects

This specification assumes that the reader has at least one year of practical experience in writing Java programs. Experience with embedded systems and server-environments is a plus. Application developers must be aware that the OSGi environment is significantly more dynamic than traditional desktop or server environments.

System developers require a very deep understanding of Java. At least three years of Java coding experience in a system environment is recommended. A Framework implementation will use areas of Java that are not normally encountered in traditional applications. Detailed understanding is required of class loaders, garbage collection, Java 2 security, and Java native library loading.

Architects should focus on the introduction of each subject. This introduction contains a general overview of the subject, the requirements that influenced its design, and a short description of its operation as well as the entities that are used. The introductory sections require knowledge of Java concepts like classes and interfaces, but should not require coding experience.

Most of these specifications are equally applicable to application developers and system developers.

1.2 Version Information

Each OSGi specification has an independent version, and packages defined by those specifications have their own versions as well. Specification and package versions follow semantic versioning rules.

1.3 References

- [1] *OSGi Specifications*
<https://docs.osgi.org/specification/>

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT

143 Network Interface Information Service Specification

Version 1.0

143.1 Introduction

The Network Interface Information Service is a service that provides a standard way for bundles to receive notification about changes in the network interface and IP address.

When the IP address has changed, bundles utilizing the IP address information need to detect this change. When using the standard Java API, such as `java.net.NetworkInterface` and `java.net.InetAddress`, calls to confirm the IP address at regular intervals are required. Since this is a process common to all bundles that need to detect any change in IP address information, this specification defines a notification feature for all available network interfaces, including the IP address. In addition, this specification defines an API that provides the function to obtain the network interface information and the information about the IP address bound to a network interface.

The name of a network interface can be Operating System specific. In order for bundles to refer to the network interface it is necessary to distinguish the network interface in a form that it is independent of the Operating System.

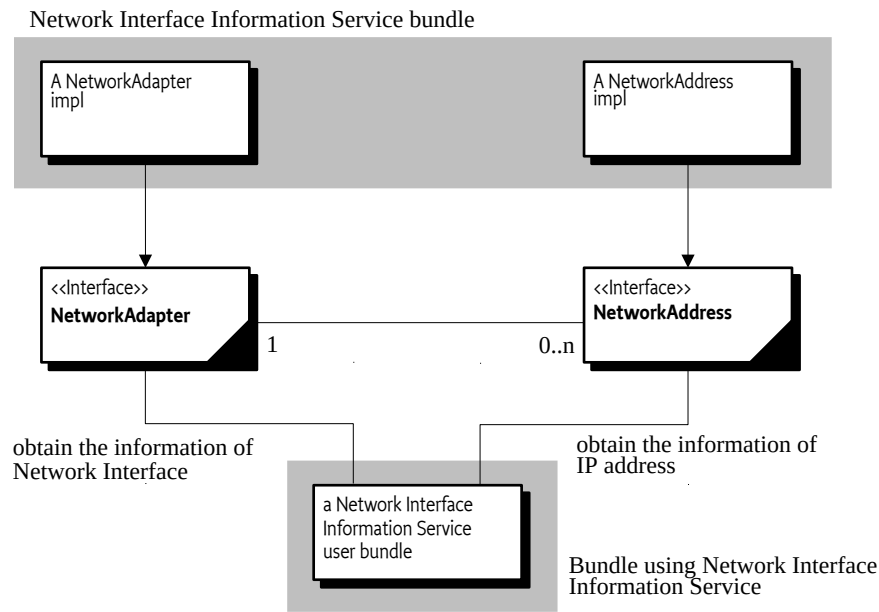
This specification defines the `NetworkAdapter` Service and `NetworkAddress` Service. These services provide information about the network interface and IP addresses.

143.1.1 Entities

- *Network Interface* - Available and activated network interfaces provided in the execution environment. In this specification, the unit of the network interface is the logical interface, not the physical interface.
- *NetworkAdapter* - The OSGi service that provides information related to the Network Interface. This service provides function similar to `java.net.NetworkInterface`.
- *NetworkAddress* - The OSGi service that provides information of IP addresses available on the execution environment in which a Network Interface Information Service bundle is running.
- *Network Interface Information Service bundle* - The OSGi bundle that implements `NetworkAdapter` and `NetworkAddress` services. Network Interface Information Service bundle registers `NetworkAdapter` and `NetworkAddress` services with the Framework.
- *Network Interface Type* - An identifier of the network interface. It is independent of the operating system. The two type of identifier string is specified in this specification. This specification allows that Network Interface type other than them can be defined by the platform provider in each environment. This identifier is used by user bundle to specify the network interface to be monitored.
- *IPAddressVersion* - An identifier indicating the IP address version. For example, IPv4, IPv6. This identifier is defined in this specification. This identifier is used by a bundle to specify the network interface to be monitored.

- *IPAddressScope* - An identifier indicating the scope of IP address. For example, GLOBAL, PRIVATE. This identifier is defined in this specification. This identifier is used by a bundle to specify the network interface to be monitored.

Figure 143.1 Network Interface Information Service Overview Diagram



The [NetworkAdapter](#) service provides the network interface information for a logical interface. [NetworkAddress](#) service provides the IP address information for an IP address. A [NetworkAddress](#) service is associated with a [NetworkAdapter](#) service.

When network interface information is changed, the service properties of the corresponding [NetworkAdapter](#) service and [NetworkAddress](#) service are changed. It is necessary for the bundle using these services to track these services and be advised of changes in the network interface information through Service Events.

143.2 NetworkAdapter Service

[NetworkAdapter](#) is an interface that provides information about a single network interface provided by the execution environment. If multiple network interfaces are present, [NetworkAdapter](#) services that correspond to each network interface must be registered. [NetworkAdapter](#) services must be registered with service properties as shown in the following table.

Table 143.1 Service properties of [NetworkAdapter](#) service

The key of service property	Type	Description
networkAdapter.type	String	Required property. Network interface type is set to a value.

The key of service property	Type	Description
networkAdapter.hardwareAddress	byte[]	Required property. Hardware address (MAC address) is set to a value. This property can also be obtained from getHardwareAddress() .
networkAdapter.name	String	Required property. Network interface name is set to a value. This property can also be obtained from getName() .
networkAdapter.displayName	String	Required property. Network interface display name is set to a value. This property can also be obtained from getDisplayName() .
networkAdapter.isUp	boolean	Required property. The value is true when a network interface is up and running, otherwise it is false.
networkAdapter.isLoopback	boolean	Required property. The value is true when a network interface is a loopback interface, otherwise it is false.
networkAdapter.isPointToPoint	boolean	Required property. The value is true when a network interface is a point to point interface, otherwise it is false.
networkAdapter.isVirtual	boolean	Required property. The value is true when a network interface is a virtual interface, otherwise it is false.
networkAdapter.supportsMulticast	boolean	Required property. The value is true when a network interface supports multicasting, otherwise it is false.
networkAdapter.parent	String	Required property. Service PID of the NetworkAdapter service which is parent of this NetworkAdapter is specified.
networkAdapter.subInterface	String[]	Required property. Service PID of the NetworkAdapter service which is subinterface of this NetworkAdapter is specified.

When a network interface becomes available, a NetworkAdapter service associated with the network interface is registered with the service registry. If the network interface becomes unavailable, the corresponding NetworkAdapter service is unregistered.

When the attribute values of the network interface change, the NetworkAdapter service is updated with changed service properties. NetworkAdapter interface provides a method corresponding to `java.net.NetworkInterface` in order to provide information on the associated network interface.

143.2.1 Network Interface Type

Identifying the network interface is possible by using the network interface name. However, since the network interface name is an identifier that is dependent on the operating system, if network interface name is used as identifier, bundles must be implemented to be aware of the operating system. Therefore, in this specification, "network interface type" which is independent of the operating system, is used to identify the network interface. The network interface type string of "LAN" and "WAN" are defined in this specification. This specification allows that Network Interface type other than "LAN" and "WAN" can be defined by the platform provider in each environment. It is provided by the platform provider on which Network Interface Information Service bundle is running. Network Interface type "LAN" indicates the network interface connects to a local area network. Network Interface type "WAN" indicates the network interface connects to an external network (i.e. Internet).

If a bundle wants to obtain the information of the network interface connected to the Internet, the bundle is able to get it by obtaining NetworkAdapter service with the networkAdapter.type service property set to the value "WAN".

This specification allows that Network Interface type other than "LAN" and "WAN" can be defined by the platform provider in each environment. It may be provided by the platform provider on which Network Interface Information Service bundle is running.

Table 143.2 Network Interface Type

Network Interface Type	Description
LAN	The network interface to connect to a local area network.
WAN	The network interface to connect to an external network (i.e. Internet).

143.3

NetworkAddress Service

NetworkAddress interface provides information about an IP address available in the execution environment in which the a Network Interface Information Service bundle is running. NetworkAddress service is registered with the service registry together with service properties as shown in the following table.

Table 143.3 Service properties of NetworkAddress service

The key of service property	Type	Description
networkAdapter.type	String	Required property. Network interface type is set to a value.
ipAddress.version	String	Required property. IP address version is set to a value.
ipAddress.scope	String	Required property. IP address scope is set to a value.
ipAddress	String	Required property. IP address String is set to a value.
subnetmask.length	int	Required property. Subnet mask length of the required properties IPv4, or IPv6 prefix length is set to a value.
networkAdapter.pid	String	Required property. Service PID of the NetworkAdapterService corresponding to the network interface binding this IP address is set to a value.

A NetworkAddress service is registered with the service registry for each available IP address. When an associated IP address is deleted, or the network interface to which the IP address is bound becomes unavailable, the NetworkAddress service is unregistered. When the associated IP address changes, the NetworkAddress service is updated with updated service properties. A bundle can detect the change of IP address by monitoring the registration or unregistering, updating of the NetworkAddress service. When registering a NetworkAdapter service, the Network Interface Information Service bundle must register it with a unique service PID. Because IP addresses are bound to a network interface, the service PID of the associated NetworkAdapter service and its network interface type are set in the service properties of the NetworkAddress service.

143.3.1

IP Address Version

Defines the version of the IP address. A bundle can select NetworkAddress services using the following IP address version.

Table 143.4 *IP Address Version*

IP Address Version	Description
IPV4	IP address version which means IPv4 address.
IPV6	IP address version which means IPv6 address.

143.3.2 IP address scope

Defines the scope of the IP address. A bundle can select NetworkAddress services using the following IP address scope.

Table 143.5 *IP Address Scope*

IP Address Scope	Description
GLOBAL	IP address scope which means global address.
PRIVATE_USE	IP address scope which means private address.
LOOPBACK	IP address scope which means loopback address.
LINKLOCAL	IP address scope which means link local address.
UNIQUE_LOCAL	IP address scope which means unique-localaddress.
UNSPECIFIED	IP address scope which means the absence of an address.

If a bundle which wants to check for an IP address of the IPv4 global, the bundle is able to confirm by obtaining NetworkAddress service with the ipAddress.version service property set to the value "IPV4" and the ipAddress.scope service property set to the value "GLOBAL".

143.4 A Controller Example

The following example shows the usage of NetworkAddress service. The sample Controller class extends the ServiceTracker class so that it can track NetworkAddress services.

```
class Controller extends ServiceTracker {
    Controller(BundleContext context) {
        super(context, NetworkAdapter.class.getName(), null);
    }

    public Object addingService(ServiceReference ref) {
        NetworkAdapter addAdapter = (NetworkAdapter)super.addingService(ref);
        String type = addAdapter.getNetworkAdapterType();
        String displayName = addAdapter.getDisplayName();

        // ...

        String servicePID = (String)ref.getProperty(Constants.SERVICE_PID);
        try {
            String filter
                = "(" + NetworkAddress.NETWORKADAPTER_PID + "=" + servicePID + ")";
            ServiceReference[] refs
                = context.getServiceReferences(NetworkAddress.class.getName(), filter);
            for (int i = 0; i < refs.length; i++) {
                NetworkAddress address = (NetworkAddress) context.getService(refs[i]);
                String ipAddress = address.getIpAddress();
                int subnetMaskLength = address.getSubnetMaskLength();
                // ...
            }
        }
    }
}
```

```

    } catch (InvalidSyntaxException e) {
        e.printStackTrace();
    }
    return addAdapter;
}
}

```

143.5 Security

To acquire network interface information, a bundle needs `ServicePermission[NetworkAdapter, GET]` and `ServicePermission[NetworkAddress, GET]`. It can use Filter Based Permissions. When a platform provider performs access control of the bundle, It can set `ServicePermission` like the following example.

```
ServicePermission["(&(objectClass=org.osgi.service.networkadapter.NetworkAdapter)(networkAdapter.type=LAN)",GET]
```

```
ServicePermission["(&(objectClass=org.osgi.service.networkadapter.NetworkAddress)(networkAdapter.type=LAN) (ipAddress.version=IPV4)(ipAddress.scope=PRIVATE_USE))", GET]
```

The `NetworkAdapter` service and the `NetworkAddress` service should only be implemented by trusted bundles. This bundle requires `ServicePermission[NetworkAdapter, REGISTER]` and `ServicePermission[NetworkAddress, REGISTER]`.

143.6 org.osgi.service.networkadapter

Network Interface Information Service Specification Package Version 1.0.

Bundles wishing to use this package must list the package in the `Import-Package` header of the bundle's manifest. This package has two types of users: the consumers that use the API in this package and the providers that implement the API in this package.

Example import for consumers using the API in this package:

```
{@code Import-Package: org.osgi.service.networkadapter; version="[1.0,2.0)"}
```

Example import for providers implementing the API in this package:

```
{@code Import-Package: org.osgi.service.networkadapter; version="[1.0,1.1)"}
```

143.6.1 Summary

- `NetworkAdapter` - `NetworkAdapter` is an interface that provides information about single network interfaces provided by the execution environment.
- `NetworkAddress` - This interface represents an IP address information.

143.6.2 public abstract NetworkAdapter

`NetworkAdapter` is an interface that provides information about single network interfaces provided by the execution environment.

If multiple network interfaces are present, `NetworkAdapter` Services that correspond to each network interface must be registered. Network interface information service is set the following information as service property.

- {@link #NETWORKADAPTER_TYPE}: Network Interface Type
- {@link #NETWORKADAPTER_DISPLAYNAME}: Network Interface Display Name

- {@link #NETWORKADAPTER_NAME}: Network Interface Name
- {@link #NETWORKADAPTER_HARDWAREADDRESS}: Hardware Address
- {@link #NETWORKADAPTER_IS_UP}: Running status of Network Interface
- {@link #NETWORKADAPTER_IS_LOOPBACK}: To check loopback interface
- {@link #NETWORKADAPTER_IS_POINTTOPOINT}: To check point to point interface
- {@link #NETWORKADAPTER_IS_VIRTUAL}: To check virtual interface
- {@link #NETWORKADAPTER_SUPPORTS_MULTICAST}: To check supports multicasting
- {@link #NETWORKADAPTER_PARENT}: The PID of parent Network Interface
- {@link #NETWORKADAPTER_SUBINTERFACE}: The PID of sub Network Interface

When a network interface becomes available, NetworkAdapter service associated with the network interface is registered with the service registry. If the network interface becomes unavailable, the corresponding NetworkAdapter service is unregistered.

When the attribute values of the network interface are set to the service property changes, NetworkAdapter service is updated. NetworkAdapter interface provides a method corresponding to java.net.NetworkInterface in order to provide information on the associated network interface. However, this interface method does not support the Static method. In addition, because NetworkInterface object or InetAddress object is registered in the service registry as NetworkAdapter and NetworkAddress, the NetworkAdapter interface does not provide a method to get those objects. NetworkAdapter provides a method to retrieve the value of an attribute of a network interface.

Concurrency Thread-safe

143.6.2.1 **public static final byte[][] EMPTY_BYTE_ARRAY**

The value byte array of service property, when information is not available.

143.6.2.2 **public static final String EMPTY_STRING = ""**

The value string of service property, when information is not available.

143.6.2.3 **public static final String[] EMPTY_STRING_ARRAY**

The value string array of service property, when information is not available.

143.6.2.4 **public static final String LAN = "LAN"**

The string of network interface type which means the network interface to connect to a local area network.

143.6.2.5 **public static final String NETWORKADAPTER_DISPLAYNAME = "networkAdapter.displayName"**

The key string of "networkAdapter.displayName" service property.

Network Interface display name is specified. {@link #EMPTY_STRING} if no display name is available.

143.6.2.6 **public static final String NETWORKADAPTER_HARDWAREADDRESS = "networkAdapter.hardwareAddress"**

The key string of "networkAdapter.hardwareAddress" service property.

Hardware Address is specified. {@link #EMPTY_BYTE_ARRAY} if no hardware address is available.

143.6.2.7 **public static final String NETWORKADAPTER_IS_LOOPBACK = "networkAdapter.isLoopback"**

The key string of "networkAdapter.isLoopback" service property.

The value is true when a network interface is a loopback interface, otherwise it is false.

143.6.2.8 **public static final String NETWORKADAPTER_IS_POINTTOPOINT = "networkAdapter.isPointToPoint"**

The key string of "networkAdapter.isPointToPoint" service property.

The value is true when a network interface is a point to point interface, otherwise it is false.

143.6.2.9 `public static final String NETWORKADAPTER_IS_UP = "networkAdapter.isUp"`

The key string of "networkAdapter.isUp" service property.

The value is true when a network interface is up and running, otherwise it is false.

143.6.2.10 `public static final String NETWORKADAPTER_IS_VIRTUAL = "networkAdapter.isVirtual"`

The key string of "networkAdapter.isVirtual" service property.

The value is true when a network interface is a virtual interface, otherwise it is false.

143.6.2.11 `public static final String NETWORKADAPTER_NAME = "networkAdapter.name"`

The key string of "networkAdapter.name" service property.

Network Interface Name is specified. {@link #EMPTY_STRING} if no name is available.

143.6.2.12 `public static final String NETWORKADAPTER_PARENT = "networkAdapter.parent"`

The key string of "networkAdapter.parent" service property.

Service PID of the NetworkAdapter service which is parent of this NetworkAdapter is specified. {@link #EMPTY_STRING} if no parent is available.

143.6.2.13 `public static final String NETWORKADAPTER_SUBINTERFACE = "networkAdapter.subInterface"`

The key string of "networkAdapter.subInterface" service property.

Service PID of the NetworkAdapter service which is subinterface of this NetworkAdapter is specified. {@link #EMPTY_STRING_ARRAY} if no subinterface is available.

143.6.2.14 `public static final String NETWORKADAPTER_SUPPORTS_MULTICAST = "networkAdapter.supportsMulticast"`

The key string of "networkAdapter.supportsMulticast" service property.

The value is true when a network interface supports multicasting, otherwise it is false.

143.6.2.15 `public static final String NETWORKADAPTER_TYPE = "networkAdapter.type"`

The key string of "networkAdapter.type" service property.

Network Interface Type is specified.

143.6.2.16 `public static final String WAN = "WAN"`

The string of network interface type which means the network interface to connect to an external network (i.e. Internet).

143.6.2.17 `public abstract String getDisplayName()`

- Returns the network interface display name of "networkAdapter.displayName" service property value.

Returns Network Interface display name, or null if "networkAdapter.displayName" service property value is empty.

143.6.2.18 `public abstract byte[][] getHardwareAddress()`

- Returns the MAC address of "networkAdapter.hardwareAddress" service property value.

Returns Hardware Address, or null if "networkAdapter.hardwareAddress" service property value is empty.

143.6.2.19 `public abstract int getMTU() throws SocketException`

- Returns the Maximum Transmission Unit (MTU) of this interface.

Returns The value of the MTU for that interface.

Throws `SocketException` – If an I/O error occurs.

143.6.2.20 **public abstract String getName()**

- Returns the network interface name of "networkAdapter.name" service property value.

Returns Network Interface Name, or null if "networkAdapter.name" service property value is empty.

143.6.2.21 **public abstract String getNetworkAdapterType()**

- Returns the network interface type of "networkAdapter.type" service property value.

Returns Network Interface Type, or null if "networkAdapter.type" service property value is empty.

143.6.2.22 **public abstract boolean isLoopback() throws SocketException**

- Returns whether a network interface is a loopback interface.

Returns true if the interface is a loopback interface.

Throws `SocketException` – If an I/O error occurs.

143.6.2.23 **public abstract boolean isPointToPoint() throws SocketException**

- Returns whether a network interface is a point to point interface.

Returns true if the interface is a point to point interface.

Throws `SocketException` – If an I/O error occurs.

143.6.2.24 **public abstract boolean isUp() throws SocketException**

- Returns whether a network interface is up and running.

Returns true if the interface is up and running.

Throws `SocketException` – If an I/O error occurs.

143.6.2.25 **public abstract boolean isVirtual()**

- Returns whether this interface is a virtual interface (also called subinterface). Virtual interfaces are, on some systems, interfaces created as a child of a physical interface and given different settings (like address or MTU). Usually the name of the interface will be the name of the parent followed by a colon (:) and a number identifying the child since there can be several virtual interfaces attached to a single physical interface.

Returns true if this interface is a virtual interface.

143.6.2.26 **public abstract boolean supportsMulticast() throws SocketException**

- Returns whether a network interface supports multicasting or not.

Returns true if the interface supports Multicasting.

Throws `SocketException` – If an I/O error occurs.

143.6.3 **public abstract NetworkAddress**

This interface represents an IP address information.

`NetworkAddress` interface provides information of IP addresses available in which execution environment on a Network Interface Information Service bundle is running. IP address information service is set the following information as service property.

- {@link #NETWORKADAPTER_TYPE}: Network Interface Type
- {@link #IPADDRESS_VERSION}: IP Address Version

- {@link #IPADDRESS_SCOPE}: IP Address Scope
- {@link #IPADDRESS}: IP Address
- {@link #SUBNETMASK_LENGTH}: Subnet Mask Length(IPv4) or Prefix Length(IPv6)
- {@link #NETWORKADAPTER_PID}: Service PID of the NetworkAdapter service to which this service belongs

NetworkAddress service is registered with the service registry for each available IP address. When associated IP addresses are deleted, or the network interface to which the IP address is bound becomes unavailable, the NetworkAddress service is unregistered. When the associated IP address changes, NetworkAddress service is updated. The user bundle can detect the change of IP address by monitoring the registration or unregistering, updating of NetworkAddress service. Because IP addresses are bound to the network interface, if any, Service PID of the associated NetworkAdapter service and its network interface type are set to service property. NetworkAdapter service MUST be registered after the all associated NetworkAddress services are registered. On the other hand, when unregistering services, after associated NetworkAdapter service is unregistered, NetworkAddress of all related services are unregistered.

Concurrency Thread-safe

143.6.3.1 **public static final Integer EMPTY_INTEGER**

The value integer of service property, when information is not available.

143.6.3.2 **public static final String IPADDRESS = "ipAddress"**

The key string of "ipAddress" service property. IP Address is specified.

143.6.3.3 **public static final String IPADDRESS_SCOPE = "ipAddress.scope"**

The key string of "ipAddress.scope" service property. IP Address scope is specified.

143.6.3.4 **public static final String IPADDRESS_SCOPE_GLOBAL = "GLOBAL"**

The string of IP address scope which means global address.

The global address is defined as the address other than the address defined in the RFC6890.

143.6.3.5 **public static final String IPADDRESS_SCOPE_HOST = "HOST"**

The string of IP address scope which means "This host on this network".

See RFC6890 for the definition of "This host on this network".

143.6.3.6 **public static final String IPADDRESS_SCOPE_LINKED_SCOPED_UNICAST = "LINKED_SCOPED_UNICAST"**

The string of IP address scope which means "Linked-Scoped Unicast".

See RFC6890 for the definition of "Linked-Scoped Unicast".

143.6.3.7 **public static final String IPADDRESS_SCOPE_LINKLOCAL = "LINKLOCAL"**

The string of IP address scope which means "Link Local".

See RFC6890 for the definition of "Link Local".

143.6.3.8 **public static final String IPADDRESS_SCOPE_LOOPBACK = "LOOPBACK"**

The string of IP address scope which means "Loopback".

See RFC6890 for the definition of "Loopback".

143.6.3.9 **public static final String IPADDRESS_SCOPE_PRIVATE_USE = "PRIVATE_USE"**

The string of IP address scope which means "Private-Use Networks".

See RFC6890 for the definition of "Private-Use Networks".

143.6.3.10 **public static final String IPADDRESS_SCOPE_SHARED = "SHARED"**

The string of IP address scope which means "Shared Address Space".

See RFC6890 for the definition of "Shared Address Space".

143.6.3.11 **public static final String IPADDRESS_SCOPE_UNIQUE_LOCAL = "UNIQUE_LOCAL"**

The string of IP address scope which means "Unique-Local".

See RFC6890 for the definition of "Unique-Local".

143.6.3.12 **public static final String IPADDRESS_SCOPE_UNSPECIFIED = "UNSPECIFIED"**

The string of IP address scope which means "Unspecified Address".

See RFC6890 for the definition of "Unspecified Address".

143.6.3.13 **public static final String IPADDRESS_VERSION = "ipAddress.version"**

The key string of "ipAddress.version" service property. IP Address version is specified.

143.6.3.14 **public static final String IPADDRESS_VERSION_4 = "IPV4"**

The string of IP address version which means IP address version 4.

143.6.3.15 **public static final String IPADDRESS_VERSION_6 = "IPV6"**

The string of IP address version which means IP address version 6.

143.6.3.16 **public static final String NETWORKADAPTER_PID = "networkAdapter.pid"**

The key string of "networkAdapter.pid" service property.

Service PID of the interface information service to which it belongs is specified.

143.6.3.17 **public static final String NETWORKADAPTER_TYPE = "networkAdapter.type"**

The key string of "networkAdapter.type" service property. Network Interface Type is specified.

143.6.3.18 **public static final String SUBNETMASK_LENGTH = "subnetmask.length"**

The key string of "subnetmask.length" service property.

Subnet Mask Length(IPv4) or Prefix Length(IPv6) is specified. {@link #EMPTY_INTEGER} if no length is available.

143.6.3.19 **public abstract InetAddress getInetAddress()**

- Returns the InetAddress object of this IP address.

Returned object is created from "ipaddress" service property value.

Returns InetAddress, or null if "ipaddress" service property value is empty.

143.6.3.20 **public abstract String getIpAddress()**

- Returns the IP address of "ipaddress" service property value.

Returns IP Address string, or null if "ipaddress" service property value is empty.

143.6.3.21 **public abstract String getIpAddressScope()**

- Returns the IP address scope of "ipaddress.scope" service property value.

Returns IP Address Scope, or null if "ipaddress.scope" service property value is empty.

143.6.3.22 public abstract String getIpAddressVersion()

- Returns the IP address version of "ipaddress.version" service property value.

Returns IP Address Version, or null if "ipaddress.version" service property value is empty.

143.6.3.23 public abstract String getNetworkAdapterPid()

- Returns the "networkadapter.pid" service property value.

Returns Service ID of the interface information service to which it belongs, or null if "networkadapter.pid" service property value is empty.

143.6.3.24 public abstract String getNetworkAdapterType()

- Returns the network interface type of "networkAdapter.type" service property value.

Returns Network Interface Type, or null if "networkAdapter.type" service property value is empty.

143.6.3.25 public abstract int getSubnetMaskLength()

- Returns the "subnetmask.length" service property value.

Returns Subnet Mask Length(IPv4) or Prefix Length(IPv6), or -1 if "subnetmask.length" service property value is empty.

143.7 References

- [1] *RFC 6890: Special-Purpose IP Address Registries*
<https://www.ietf.org/rfc/rfc6890.txt> , April 2013

End Of Document

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT